OPERATOR LAB · ORACLE RAC 19C ON KUBEADM

The operator lab: Phases 0–8

A second, separate lab next to the manual one, built the same way up to Multus, then handed to Oracle's database operator for the RAC database. Own VM names, IPs, ports and disks; the same golden image.

`op-controlplane · op-worker1 · op-worker2` `10.0.1.20–22` `SSH 2220–2222` `D:\VMs\rac-op`

`Oracle Database Operator v2.2.0`

26 Oct 2026 · Companion to the manual lab guide; Phases 7 onward follow oracle/oracle-database-operator.

`PowerShell / Git Bash`

on your PC

`golden VM · controlplane · worker`

root shell on a VM

`racnode1-0 · racnode2-0`

root shell in a RAC pod

How to use this guide

This guide builds a **second lab, the operator lab**, next to the manual lab you already have. It follows the manual guide's Phases 0 and 3–6 step by step, with its own names, addresses and ports, so the two labs never overwrite each other. The RAC database itself will come from Oracle's database operator in the later phases, instead of the hand-run installs of the manual lab's Phases 8–14.

Copy every block exactly as it is. Each block's label says where it runs, and the **connect** line shows how to get there. Blocks labelled `# PowerShell` run in a PowerShell window on your PC, never in Git Bash.

	MANUAL LAB (UNCHANGED)	OPERATOR LAB (THIS GUIDE)
VM names and host names	`controlplane`, `worker1`, `worker2`	`op-controlplane`, `op-worker1`, `op-worker2`
IPs on `k8s-public` 10.0.1.0/24	.10, .11, .12	**.20, .21, .22**
SSH from Windows	2210, 2211, 2212	**2220, 2221, 2222**
VM and disk folder	`D:\VMs\rac-poc`	`D:\VMs\rac-op`
Shared ASM disks	`rac-poc\shared\asm1.vdi`, `asm2.vdi`	`rac-op\shared\asm1.vdi`, `asm2.vdi` (new, empty)

	MANUAL LAB (UNCHANGED)	OPERATOR LAB (THIS GUIDE)
Private networks (VirtualBox internal)	`rac-priv1`, `rac-priv2`	`op-priv1`, `op-priv2`, same subnets 10.0.11/12.0/24
Kubernetes cluster name	`rac-poc`	`rac-op`
Golden VM `ol98-golden` (2205, 10.0.1.5)	shared	shared: only the clone source; nothing changes in it
NAT Network `k8s-public`	shared	shared: the operator lab only adds its own SSH forwards

Skipped phases: Phases 1 and 2 built the golden VM, which both labs share. Phase 7 (ASM disks as Kubernetes volumes) and everything after it depend on what the operator expects, so they come after reading Oracle's operator documentation.

One lab at a time. Each lab needs about 48 GB of the PC's 64 GB, and both use the same NAT Network. Before you start the operator lab, the manual lab must be powered off, and the other way round:

PowerShell

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
& $VB list runningvms
```

Check: before working on the operator lab, nothing is listed, or only `op-*` VMs.

Switching labs later (after both are built):

PowerShell

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
# stop the manual lab (stop Grid in its pods first, as in "how to shut down")
foreach ($n in "worker2","worker1","controlplane") { & $VB controlvm $n acpipowerbutton 2>$null }
# start the operator lab
foreach ($n in "op-controlplane","op-worker1","op-worker2") { & $VB startvm $n --type headless }
```

To go back, swap the two lists: stop `op-worker2`, `op-worker1`, `op-controlplane`, then start `controlplane`, `worker1`, `worker2`.

PHASE 0

Windows host and the operator lab's port forwards

Result of this phase: a clean starting point for the operator lab, the PC checked for room, and SSH port forwards 2220–2222 on the shared `k8s-public` network. The manual lab and the golden VM are not touched.

0.0 Start clean (only if you built the operator lab before)

What: removes an earlier operator lab: the VMs `op-controlplane`, `op-worker1`, `op-worker2`, the folder `D:\VMs\rac-op` with all its disks, the operator lab's port-forward rules, and the SSH host keys remembered for ports 2220–2222. On a first build it just reports that there's nothing to remove.

What it never touches: `ol98-golden`, `controlplane`, `worker1`, `worker2`, `D:\VMs\rac-poc` and the `k8s-public` network itself. Every name in the block starts with `op-` or `rac-op`.

```PowerShell
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"; $Base = "D:\VMs\rac-op"
$existing = (& $VB list vms) -join "`n"
foreach ($v in "op-worker1","op-worker2","op-controlplane") {
  if ($existing -match ('"' + $v + '"')) { & $VB controlvm $v poweroff 2>$null; Start-Sleep 3; & $VB
unregistervm $v --delete; "removed VM $v" }
}
$left = (& $VB list hdds) | Select-String '^Location:\s+(.*)$' | ForEach-Object {
$_.Matches[0].Groups[1].Value } | Where-Object { $_ -like "$Base\*" }
foreach ($d in $left) { & $VB closemedium disk $d --delete; "removed disk $d" }
if (Test-Path $Base) { Remove-Item $Base -Recurse -Force; "removed folder $Base" }
foreach ($r in "ssh-opcp","ssh-opw1","ssh-opw2","k8sapi-op") { & $VB natnetwork modify --netname k8s-
public --port-forward-4 delete $r 2>$null }
foreach ($f in "$env:USERPROFILE\.ssh\known_hosts", "$env:USERPROFILE\.ssh\known_hosts_rac_lab") {
  if (Test-Path $f) { foreach ($p in 2220, 2221, 2222) { ssh-keygen -R "[127.0.0.1]:$p" -f $f 2>$null |
Out-Null } }
}
"--- VMs (the manual lab and ol98-golden must still be listed; no op-*) ---"
& $VB list vms
```

Check: `list vms` still shows `ol98-golden`, `controlplane`, `worker1` and `worker2`, and no `op-*` VM.

0.1 Check the host

Why: the operator lab needs the same memory and CPU as the manual lab, about 48 GB, so only one lab can run at a time. On disk it needs its own room: two fixed 50 GB ASM disks are written in full straight away, and the workers' disks grow as they fill.

```PowerShell
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
& $VB list runningvms
Get-CimInstance Win32_Processor | Select-Object NumberOfCores, NumberOfLogicalProcessors
[Math]::Round((Get-CimInstance Win32_ComputerSystem).TotalPhysicalMemory / 1GB)
Get-PSDrive D | Select-Object @{n='FreeGB';e={[math]::Round($_.Free/1GB)}}
```

Check:

- `list runningvms` prints **nothing**: the manual lab and the golden VM are off;
- 8 or more logical processors and about 64 GB of RAM;
- **300 GB or more free on D:**. Less than about 150 GB stops you at 3.3, when the ASM disks are created.

0.2 SSH key and lab SSH settings (already done)

The manual lab's Phase 0.2 set these up, and they cover every port on `127.0.0.1`, including 2220–2222. This block only checks them; it changes nothing that is already there.

```powershell
if (-not (Test-Path "$env:USERPROFILE\.ssh\id_ed25519")) { ssh-keygen -t ed25519 }   # press Enter at every prompt
Select-String -Path "$env:USERPROFILE\.ssh\config" -Pattern '^Host 127\.0\.0\.1$' -Context 0,5
```

Check: the `Host 127.0.0.1` section with its five settings (`User root`, `StrictHostKeyChecking no`, `UserKnownHostsFile ~/.ssh/known_hosts_rac_lab`, `LogLevel ERROR`, `ServerAliveInterval 30`). If nothing prints, run the manual guide's 0.2 block once.

0.3 Port forwards for the operator lab

What: three SSH forwards on the existing `k8s-public` network: 2220 → op-controlplane, 2221 → op-worker1, 2222 → op-worker2. The golden VM's forward, 2205 → 10.0.1.5, is already there and is reused for personalising the clones in Phase 4.

Why on the same network: the two labs never run at the same time, so they can share `k8s-public` and its NAT. Different IPs (.20–.22) and different ports keep them apart; the rule names start with `ssh-op` so 0.0 can remove exactly these.

```powershell
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
& $VB natnetwork modify --netname k8s-public --port-forward-4 "ssh-opcp:tcp:[]:2220:[10.0.1.20]:22"
& $VB natnetwork modify --netname k8s-public --port-forward-4 "ssh-opw1:tcp:[]:2221:[10.0.1.21]:22"
& $VB natnetwork modify --netname k8s-public --port-forward-4 "ssh-opw2:tcp:[]:2222:[10.0.1.22]:22"
& $VB natnetwork list
```

Check: `natnetwork list` shows `k8s-public` with the manual lab's rules (`ssh-golden`, `ssh-cp`, `ssh-w1`, `ssh-w2`, and any NodePort rules) **plus** `ssh-opcp`, `ssh-opw1` and `ssh-opw2`. If a line says the rule already exists, it was added before; that's fine.

PHASE 3

Clone the three operator-lab nodes

Result of this phase: three VMs (`op-controlplane`, `op-worker1`, `op-worker2`) cloned from the same golden image as the manual lab, the workers with two data disks each and their private NICs on `op-priv1` / `op-priv2`, and two new 50 GB disks that **both** workers share for ASM.

3.1 Check that the golden VM is still sealed

Why: clones must come from the **sealed** golden image: no machine ID, no SSH host keys. You sealed it before cloning the manual lab. If the golden VM has been started since then, it has made itself a new machine ID and host keys, and every clone would share them. Kubernetes refuses to join nodes with the same machine ID.

How to tell without starting it: VirtualBox records when a VM last changed state. If the golden VM's last change (its power-off after sealing) is **older** than the manual lab's clones, nobody has started it since.

PowerShell

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"; $g = "ol98-golden"
$info = & $VB showvminfo $g --machinereadable
$state = ($info | Select-String '^VMState="(.*)"').Matches[0].Groups[1].Value
$raw = ($info | Select-String '^VMStateChangeTime="(.*)"').Matches[0].Groups[1].Value
$changed = [datetime]::SpecifyKind([datetime]::ParseExact($raw.Substring(0,19), 'yyyy-MM-ddTHH:mm:ss',
$null), 'Utc')   # VirtualBox writes UTC
$cpCfg = (& $VB showvminfo controlplane --machinereadable | Select-String '^CfgFile="
(.*)"').Matches[0].Groups[1].Value
$cloned = (Get-Item (Split-Path $cpCfg)).CreationTimeUtc   # when the manual lab's controlplane was
cloned
"manual lab folder:    $(Split-Path $cpCfg)"
"golden state:         $state"
"golden last changed:  $($changed.ToString('u'))"
"manual lab cloned:    $($cloned.ToString('u'))"
if ($state -eq 'poweroff' -and $changed -lt $cloned) { "OK: still sealed, not started since the manual
lab was cloned" } else { "STOP: golden VM was started after the seal; reseal it (below)" }
& $VB snapshot $g list
```

Check: the line says `OK: still sealed…`, and the snapshot list shows the `pre-seal` snapshot. Then go straight to 3.2. The block finds the manual lab's folder from VirtualBox itself, wherever it is on your disk.

Only if it says STOP: seal it again, in place. Don't restore the `pre-seal` snapshot: fixes made to the golden image after that snapshot would be lost. Starting the golden VM gives it a new machine ID and host keys for a moment; the seal block removes them again and powers it off.

PowerShell (only after a STOP in the check above)

```
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" startvm ol98-golden --type headless
ssh-keygen -R "[127.0.0.1]:2205" 2>$null
```

golden VM

(the last command powers it off; don't start it again before cloning)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
dnf clean all
rm -f /etc/ssh/ssh_host_*
truncate -s 0 /etc/machine-id
rm -f /var/lib/dbus/machine-id
rm -rf /tmp/* /var/tmp/*
journalctl --rotate; journalctl --vacuum-time=1s
rm -f /root/.bash_history; history -c
poweroff
```

If SSH warns that the host key for port 2205 has changed, run `ssh-keygen -R "[127.0.0.1]:2205"` in PowerShell and connect again.

3.2 Clone and size the nodes

Why these sizes: the same as the manual lab. op-controlplane gets 2 vCPU and 4 GB with no private NICs; each worker gets 4 vCPU and 22 GB, an 80 GB disk for `/scratch` and a 100 GB disk for `/var/lib/containers`.

What's different: the workers' second and third NICs move from the manual lab's internal networks `rac-priv1` / `rac-priv2` to the operator lab's `op-priv1` / `op-priv2`. The clones inherit the golden VM's NIC settings, including the "Allow All" promiscuous mode Multus needs; only the network names change.

PowerShell

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"; $Base = "D:\VMs\rac-op"; $g = "ol98-golden"
while ("$(& $VB showvminfo $g --machinereadable | Select-String '^VMState=')" -notmatch 'poweroff') {
Start-Sleep 3 }
foreach ($n in "op-controlplane","op-worker1","op-worker2") { & $VB clonevm $g --name $n --basefolder
$Base --mode machine --register }
& $VB modifyvm op-controlplane --cpus 2 --memory 4096 --nic2 none --nic3 none
foreach ($w in "op-worker1","op-worker2") {
  & $VB modifyvm $w --cpus 4 --memory 22528 --intnet2 op-priv1 --intnet3 op-priv2
  & $VB createmedium disk --filename "$Base\$w\$w-scratch.vdi" --size 81920 --variant Standard
  & $VB createmedium disk --filename "$Base\$w\$w-containers.vdi" --size 102400 --variant Standard
  & $VB storageattach $w --storagectl SATA --port 1 --device 0 --type hdd --medium "$Base\$w\$w-
scratch.vdi"
  & $VB storageattach $w --storagectl SATA --port 2 --device 0 --type hdd --medium "$Base\$w\$w-
containers.vdi"
}
foreach ($n in "op-controlplane","op-worker1","op-worker2") { "--- $n"; & $VB showvminfo $n --
machinereadable | Select-String '^(cpus|memory|nic[123]|nat-network1|intnet[23])='; & $VB showvminfo $n |
Select-String '^NIC [23]:' }
```

Check:

- **op-controlplane:** `cpus=2`, `memory=4096`, `nic1="natnetwork"` with `nat-network1="k8s-public"`, and `nic2="none"`, `nic3="none"`;
- **each worker:** `cpus=4`, `memory=22528`, `nat-network1="k8s-public"`, `intnet2="op-priv1"`, `intnet3="op-priv2"`, and two `NIC 2:` / `NIC 3:` lines that include `Promisc Policy: allow-all`, each with its own MAC address. (The machine-readable output has no promiscuous-mode line on this VirtualBox version, so the check reads it from the normal output.)

3.3 The shared ASM disks

What and why: the same as the manual lab: two fixed-size 50 GB disks, attached to **both** workers as `shareable`, because RAC needs the same block devices on both nodes. These are **new, empty** disks in `D:\VMs\rac-op\shared`; the manual lab's disks stay with the manual lab. Don't snapshot the workers: VirtualBox can't snapshot a VM with shareable disks.

PowerShell

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"; $Base = "D:\VMs\rac-op"
New-Item -ItemType Directory -Force -Path "$Base\shared" | Out-Null
$port = 3
foreach ($d in "asm1","asm2") {
  & $VB createmedium disk --filename "$Base\shared\$d.vdi" --size 51200 --variant Fixed
  & $VB modifymedium disk "$Base\shared\$d.vdi" --type shareable
  foreach ($w in "op-worker1","op-worker2") { & $VB storageattach $w --storagectl SATA --port $port --
device 0 --type hdd --medium "$Base\shared\$d.vdi" --mtype shareable }
  $port++
}
foreach ($d in "asm1","asm2") { & $VB showmediuminfo disk "$Base\shared\$d.vdi" | Select-String
'^(UUID|Location|Type|In use by VMs):|op-worker' }
```

Check: for **each** disk, `Type: shareable`, a `Location` under `D:\VMs\rac-op\shared`, and both `op-worker1` and `op-worker2` under "In use by VMs". Note the first 8 characters of each UUID: inside the workers the disk appears as `/dev/disk/by-id/ata-VBOX_HARDDISK_VB<those 8>-...`.

PHASE 4

Personalize each node

Result of this phase: each node has its own name and IP address. The workers also have their private NICs ready for Multus, `/scratch` and `/var/lib/containers` on their own disks, the operator's directories, and 4 GB of hugepages.

Do one node at a time, in the order op-controlplane, op-worker1, op-worker2. Every clone boots with the golden VM's address 10.0.1.5, and two clones running at once would fight over it. Each node has its own section below with every command written out for it: copy the blocks in order, without changing anything. Start the next node only after the previous one passes its last check.

4.1 What happens to each node, and why

STEP	OP-CONTROLPLANE	OP-WORKER1, OP-WORKER2
a. Start and connect on 2205	yes	yes
b. Name, identity, kubelet IP	yes	yes
c. Private NICs for Multus	no: it has no NIC2/NIC3	yes
d, e. Data disks `/var/lib/containers` and `/scratch`	no: it has only the OS disk	yes
f. The operator's directories	no	yes (the same on both workers)

STEP	OP-CONTROLPLANE	OP-WORKER1, OP-WORKER2
g. HugePages	no: 2048 pages would lock all of its 4 GB	yes
Move to its own address, then check it	yes	yes

Why each step:

- **Name, identity and kubelet IP:** the host name becomes the Kubernetes node name. New SSH host keys replace the ones the seal removed. `--node-ip` matters on the workers, which have three NICs: without it the kubelet may advertise the wrong address, and pod traffic between nodes breaks.
- **Private NICs with no IP:** `enp0s8` and `enp0s9` are only "parents" for Multus macvlan interfaces. Each RAC pod gets its own interface, IP (10.0.11.x and 10.0.12.x) and MAC address on top of them.
- **Promiscuous mode (** `accept-all-mac-addresses yes` **):** this was the cause of the hardest network failure in the first build, 100% packet loss between pods. VirtualBox's "Allow All" setting only takes effect when the guest NIC itself is promiscuous. Otherwise the virtual NIC silently drops frames addressed to the pods' macvlan MACs.
- **Data disks chosen by size, not by name:** Linux disk names (`sdb` , `sdc` ...) do **not** follow VirtualBox port order. On worker2 in the manual lab's first build, `/dev/sdc` was a shared ASM disk, and formatting by name would have wiped the disk both nodes share. The ASM disks are 50 GB, so the sizes tell the disks apart. They're **mounted by UUID** for the same reason, and `rsync -aAXH` copies the images pulled in Phase 2 together with their SELinux labels.
- **The operator's directories with** `container_file_t` **:** the operator mounts them into the RAC pods as hostPath volumes; their names follow its `racHostSwLocation` (`/scratch/rac/cluster01`) and `racNodeName` (`racnode`) settings. With SELinux enforcing, a container can only use host files with the container file label. `semanage fcontext` makes the label permanent.
- **HugePages:** the database SGA goes into 2 MB hugepages, which are locked, never swapped and cheaper for the CPU to map. Kubernetes only gives hugepages to pods on nodes that reserved them: the RAC pods ask for `hugepages-2Mi: 4Gi` , so each worker reserves 2048 pages. The doc reserves 16384 (32 GB); this lab scales it to the 22 GB workers.
- **Moving the address last, in the background:** changing the public IP cuts the SSH session you're typing in. A first attempt died halfway through for exactly that reason. So the change runs detached from your session and ends with a reboot.
- `ssh-keygen -R` **before connecting:** every clone has its own host key but answers on the same port, 2205. Without the 0.2 SSH settings, SSH refuses the new key with "REMOTE HOST IDENTIFICATION HAS CHANGED"; removing the old entry avoids that.

4.2 op-controlplane

a. Start it and connect (port 2205 reaches whichever clone has 10.0.1.5):

PowerShell

```
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" startvm op-controlplane --type headless
ssh-keygen -R "[127.0.0.1]:2205" 2>$null
ssh -p 2205 root@127.0.0.1
```

If SSH says "connection refused" or times out, the VM is still booting: wait 30 seconds and run the `ssh` line again. Answer `yes` to the host key question. **Check:** `hostname` says `ol98-golden` .

b. Name, identity and kubelet IP:

`op-controlplane` (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
hostnamectl set-hostname op-controlplane
[ -s /etc/machine-id ] || systemd-machine-id-setup
ssh-keygen -A
echo 'KUBELET_EXTRA_ARGS="--node-ip=10.0.1.20"' >/etc/sysconfig/kubelet
echo "machine-id $(cat /etc/machine-id)  product_uuid $(cat /sys/class/dmi/id/product_uuid)"
hostname; cat /etc/sysconfig/kubelet
```

Check: `op-controlplane`, `--node-ip=10.0.1.20`, and a machine ID and product UUID different from the other nodes'.

c. Move to 10.0.1.20 and reboot (always last):

`op-controlplane` (still on 10.0.1.5; the session freezes or closes at the end)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
if [ "$(hostname)" = "op-controlplane" ]; then
cat >/root/set-ip.sh <<'EOF'
for u in $(nmcli -g UUID con show); do
  [ "$(nmcli -g connection.interface-name con show $u)" = "enp0s3" ] && nmcli con delete $u
done
nmcli con add type ethernet con-name pub ifname enp0s3 ipv4.method manual ipv4.addresses 10.0.1.20/24 \
  ipv4.gateway 10.0.1.1 ipv4.dns 8.8.8.8 ipv6.method disabled connection.autoconnect yes
sleep 2; reboot
EOF
setsid nohup bash /root/set-ip.sh >/root/set-ip.log 2>&1 </dev/null &
else echo "STOP: this VM is $(hostname), not op-controlplane; run step b first"; fi
```

Your SSH session freezes or closes. That's expected: the VM is moving to 10.0.1.20 and rebooting.

d. Check it on its own address (wait about a minute after the move):

`PowerShell`

```
ssh -p 2220 root@127.0.0.1 "hostname; ip -4 -br addr show enp0s3; ip -br link show enp0s8; df -h /scratch
/var/lib/containers; grep HugePages_Total /proc/meminfo; systemctl is-active crio"
```

Check:

- `op-controlplane`, and `10.0.1.20/24`;
- `Device "enp0s8" does not exist` and `df: /scratch: No such file or directory`: correct, it has no private NICs and no data disks;
- `/var/lib/containers` is on the root filesystem (`/dev/mapper/ol-root`);
- `HugePages_Total: 0`, and CRI-O is `active`.

4.3 op-worker1

a. Start it and connect (port 2205 reaches whichever clone has 10.0.1.5):

PowerShell

```
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" startvm op-worker1 --type headless
ssh-keygen -R "[127.0.0.1]:2205" 2>$null
ssh -p 2205 root@127.0.0.1
```

If SSH says "connection refused" or times out, the VM is still booting: wait 30 seconds and run the `ssh` line again. Answer `yes` to the host key question. **Check:** `hostname` says `ol98-golden`.

b. Name, identity and kubelet IP:

op-worker1 (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
hostnamectl set-hostname op-worker1
[ -s /etc/machine-id ] || systemd-machine-id-setup
ssh-keygen -A
echo 'KUBELET_EXTRA_ARGS="--node-ip=10.0.1.21"' >/etc/sysconfig/kubelet
echo "machine-id $(cat /etc/machine-id)  product_uuid $(cat /sys/class/dmi/id/product_uuid)"
hostname; cat /etc/sysconfig/kubelet
```

Check: `op-worker1`, `--node-ip=10.0.1.21`, and a machine ID and product UUID different from the other nodes'.

c. Private NICs, up with no IP and promiscuous:

op-worker1 (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
for ifn in enp0s8 enp0s9; do
  for u in $(nmcli -g UUID con show); do
    [ "$(nmcli -g connection.interface-name con show $u)" = "$ifn" ] && nmcli con delete $u
  done
  nmcli con add type ethernet con-name rac-$ifn ifname $ifn ipv4.method disabled ipv6.method disabled \
    802-3-ethernet.mtu 1500 802-3-ethernet.accept-all-mac-addresses yes connection.autoconnect yes
done
ip -br link show enp0s8; ip -br link show enp0s9
```

Check: both NICs show `UP` and have no IPv4 address. `PROMISC` shows after the reboot in step h.

d. Find the two data disks by size:

op-worker1 (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
lsblk -d -o NAME,SIZE,TYPE
pick() { lsblk -dbno NAME,SIZE,TYPE | awk -v lo=$(( ($1-2)*1024**3 )) -v hi=$(( ($1+2)*1024**3 ))
'$3=="disk" && $2>=lo && $2<=hi {print "/dev/"$1}'; }
CONT=$(pick 100); SCRATCH=$(pick 80); echo "containers=$CONT scratch=$SCRATCH"
[ $(echo "$CONT" | wc -w) -eq 1 ] && [ $(echo "$SCRATCH" | wc -w) -eq 1 ] && [ -z "$(blkid -s TYPE -o
value $CONT $SCRATCH)" ] && echo "OK: two blank disks found" || echo "STOP: check lsblk before
formatting"
```

Check: it prints `OK: two blank disks found`. If it prints STOP, don't go on; paste the `lsblk` output.

e. Format and mount them (same session as d; it formats nothing unless d found the two blank disks):

op-worker1 (still on 10.0.1.5, same session as step d)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
if [ -b "$CONT" ] && [ -b "$SCRATCH" ] && [ "$CONT" != "$SCRATCH" ] && [ -z "$(blkid -s TYPE -o value
$CONT $SCRATCH)" ]; then
systemctl stop kubelet crio
mkfs.xfs -q -f -L containers $CONT
mkdir -p /mnt/newcont && mount $CONT /mnt/newcont
rsync -aAXH /var/lib/containers/ /mnt/newcont/
umount /mnt/newcont && rmdir /mnt/newcont
echo "UUID=$(blkid -o value -s UUID $CONT)  /var/lib/containers  xfs  defaults  0 0" >>/etc/fstab
mkfs.xfs -q -f -L scratch $SCRATCH
mkdir -p /scratch
echo "UUID=$(blkid -o value -s UUID $SCRATCH)  /scratch  xfs  defaults  0 0" >>/etc/fstab
systemctl daemon-reload
mount /var/lib/containers && mount /scratch
restorecon -R /var/lib/containers
df -h /scratch /var/lib/containers
else echo "STOP: nothing formatted. Run step d in this same session; it must print OK"; fi
```

Check: `/scratch` about 80 G and `/var/lib/containers` about 100 G.

f. The operator's directories, with SELinux labels: `/scratch/rac/cluster01/racnode1` and `/scratch/rac/cluster01/racnode2` (the operator mounts one of them into each RAC pod as `/u01`), and `/scratch/software/stage` (the Oracle zips). **Both workers get both node directories:** the operator may place either RAC pod on either labelled worker.

`op-worker1` (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
for d in /scratch/rac/cluster01/racnode1 /scratch/rac/cluster01/racnode2 /scratch/software/stage; do
mkdir -p $d; done
chown -R 54321:54321 /scratch/rac/cluster01
semanage fcontext -a -t container_file_t "/scratch/rac/cluster01(/.*)?"
semanage fcontext -a -t container_file_t "/scratch/software/stage(/.*)?"
restorecon -RF /scratch/rac/cluster01 /scratch/software/stage
ls -Zd /scratch/rac/cluster01/racnode1 /scratch/rac/cluster01/racnode2 /scratch/software/stage; ls -ld
/scratch/rac/cluster01/racnode*
```

Check: all three directories show `container_file_t`, and both `racnode*` directories are owned by `54321 54321`, shown as `oracle oinstall` if the host has those users: the IDs of `oracle` and `oinstall` inside the RAC image.

g. HugePages:

`op-worker1` (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
echo "vm.nr_hugepages = 2048" >/etc/sysctl.d/92-hugepages.conf
sysctl -p /etc/sysctl.d/92-hugepages.conf
grep -E 'HugePages_Total|Hugepagesize' /proc/meminfo
```

Check: `HugePages_Total: 2048` and `Hugepagesize: 2048 kB`.

h. Move to 10.0.1.21 and reboot (always last):

`op-worker1` (still on 10.0.1.5; the session freezes or closes at the end)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
if [ "$(hostname)" = "op-worker1" ]; then
cat >/root/set-ip.sh <<'EOF'
for u in $(nmcli -g UUID con show); do
  [ "$(nmcli -g connection.interface-name con show $u)" = "enp0s3" ] && nmcli con delete $u
done
nmcli con add type ethernet con-name pub ifname enp0s3 ipv4.method manual ipv4.addresses 10.0.1.21/24 \
  ipv4.gateway 10.0.1.1 ipv4.dns 8.8.8.8 ipv6.method disabled connection.autoconnect yes
sleep 2; reboot
EOF
setsid nohup bash /root/set-ip.sh >/root/set-ip.log 2>&1 </dev/null &
else echo "STOP: this VM is $(hostname), not op-worker1; run step b first"; fi
```

Your SSH session freezes or closes. That's expected: the VM is moving to 10.0.1.21 and rebooting.

i. Check it on its own address (wait about a minute after the move):

PowerShell

```
ssh -p 2221 root@127.0.0.1 "hostname; ip -4 -br addr show enp0s3; ip -br link show enp0s8; df -h /scratch
/var/lib/containers; grep HugePages_Total /proc/meminfo; systemctl is-active crio"
```

Check:

- `op-worker1`, and `10.0.1.21/24`;
- `enp0s8` shows `UP` and `PROMISC`, with no address;
- `/scratch` is 80 G, and `/var/lib/containers` is 100 G;
- `HugePages_Total: 2048`;
- CRI-O is `active`.

4.4 op-worker2

a. Start it and connect (port 2205 reaches whichever clone has 10.0.1.5):

PowerShell

```
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" startvm op-worker2 --type headless
ssh-keygen -R "[127.0.0.1]:2205" 2>$null
ssh -p 2205 root@127.0.0.1
```

If SSH says "connection refused" or times out, the VM is still booting: wait 30 seconds and run the `ssh` line again. Answer `yes` to the host key question. **Check:** `hostname` says `ol98-golden`.

b. Name, identity and kubelet IP:

op-worker2 (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
hostnamectl set-hostname op-worker2
[ -s /etc/machine-id ] || systemd-machine-id-setup
ssh-keygen -A
echo 'KUBELET_EXTRA_ARGS="--node-ip=10.0.1.22"' >/etc/sysconfig/kubelet
echo "machine-id $(cat /etc/machine-id)  product_uuid $(cat /sys/class/dmi/id/product_uuid)"
hostname; cat /etc/sysconfig/kubelet
```

Check: `op-worker2`, `--node-ip=10.0.1.22`, and a machine ID and product UUID different from the other nodes'.

c. Private NICs, up with no IP and promiscuous:

`op-worker2` (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
for ifn in enp0s8 enp0s9; do
  for u in $(nmcli -g UUID con show); do
    [ "$(nmcli -g connection.interface-name con show $u)" = "$ifn" ] && nmcli con delete $u
  done
  nmcli con add type ethernet con-name rac-$ifn ifname $ifn ipv4.method disabled ipv6.method disabled \
    802-3-ethernet.mtu 1500 802-3-ethernet.accept-all-mac-addresses yes connection.autoconnect yes
done
ip -br link show enp0s8; ip -br link show enp0s9
```

Check: both NICs show `UP` and have no IPv4 address. `PROMISC` shows after the reboot in step h.

d. Find the two data disks by size:

`op-worker2` (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
lsblk -d -o NAME,SIZE,TYPE
pick() { lsblk -dbno NAME,SIZE,TYPE | awk -v lo=$(( ($1-2)*1024**3 )) -v hi=$(( ($1+2)*1024**3 ))
'$3=="disk" && $2>=lo && $2<=hi {print "/dev/"$1}'; }
CONT=$(pick 100); SCRATCH=$(pick 80); echo "containers=$CONT scratch=$SCRATCH"
[ $(echo "$CONT" | wc -w) -eq 1 ] && [ $(echo "$SCRATCH" | wc -w) -eq 1 ] && [ -z "$(blkid -s TYPE -o
value $CONT $SCRATCH)" ] && echo "OK: two blank disks found" || echo "STOP: check lsblk before
formatting"
```

Check: it prints `OK: two blank disks found`. If it prints STOP, don't go on; paste the `lsblk` output.

e. Format and mount them (same session as d; it formats nothing unless d found the two blank disks):

`op-worker2` (still on 10.0.1.5, same session as step d)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
if [ -b "$CONT" ] && [ -b "$SCRATCH" ] && [ "$CONT" != "$SCRATCH" ] && [ -z "$(blkid -s TYPE -o value
$CONT $SCRATCH)" ]; then
systemctl stop kubelet crio
mkfs.xfs -q -f -L containers $CONT
mkdir -p /mnt/newcont && mount $CONT /mnt/newcont
rsync -aAXH /var/lib/containers/ /mnt/newcont/
umount /mnt/newcont && rmdir /mnt/newcont
echo "UUID=$(blkid -o value -s UUID $CONT)  /var/lib/containers  xfs  defaults  0 0" >>/etc/fstab
mkfs.xfs -q -f -L scratch $SCRATCH
mkdir -p /scratch
echo "UUID=$(blkid -o value -s UUID $SCRATCH)  /scratch  xfs  defaults  0 0" >>/etc/fstab
systemctl daemon-reload
mount /var/lib/containers && mount /scratch
restorecon -R /var/lib/containers
df -h /scratch /var/lib/containers
else echo "STOP: nothing formatted. Run step d in this same session; it must print OK"; fi
```

Check: `/scratch` about 80 G and `/var/lib/containers` about 100 G.

f. The operator's directories, with SELinux labels: `/scratch/rac/cluster01/racnode1` and `/scratch/rac/cluster01/racnode2` (the operator mounts one of them into each RAC pod as `/u01`), and `/scratch/software/stage` (the Oracle zips). **Both workers get both node directories:** the operator may place either RAC pod on either labelled worker.

op-worker2 (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
for d in /scratch/rac/cluster01/racnode1 /scratch/rac/cluster01/racnode2 /scratch/software/stage; do
mkdir -p $d; done
chown -R 54321:54321 /scratch/rac/cluster01
semanage fcontext -a -t container_file_t "/scratch/rac/cluster01(/.*)?"
semanage fcontext -a -t container_file_t "/scratch/software/stage(/.*)?"
restorecon -RF /scratch/rac/cluster01 /scratch/software/stage
ls -Zd /scratch/rac/cluster01/racnode1 /scratch/rac/cluster01/racnode2 /scratch/software/stage; ls -ld
/scratch/rac/cluster01/racnode*
```

Check: all three directories show `container_file_t`, and both `racnode*` directories are owned by `54321 54321`, shown as `oracle oinstall` if the host has those users: the IDs of `oracle` and `oinstall` inside the RAC image.

g. HugePages:

op-worker2 (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
echo "vm.nr_hugepages = 2048" >/etc/sysctl.d/92-hugepages.conf
sysctl -p /etc/sysctl.d/92-hugepages.conf
grep -E 'HugePages_Total|Hugepagesize' /proc/meminfo
```

Check: `HugePages_Total: 2048` and `Hugepagesize: 2048 kB`.

h. Move to 10.0.1.22 and reboot (always last):

op-worker2 (still on 10.0.1.5; the session freezes or closes at the end)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
if [ "$(hostname)" = "op-worker2" ]; then
cat >/root/set-ip.sh <<'EOF'
for u in $(nmcli -g UUID con show); do
  [ "$(nmcli -g connection.interface-name con show $u)" = "enp0s3" ] && nmcli con delete $u
done
nmcli con add type ethernet con-name pub ifname enp0s3 ipv4.method manual ipv4.addresses 10.0.1.22/24 \
  ipv4.gateway 10.0.1.1 ipv4.dns 8.8.8.8 ipv6.method disabled connection.autoconnect yes
sleep 2; reboot
EOF
setsid nohup bash /root/set-ip.sh >/root/set-ip.log 2>&1 </dev/null &
else echo "STOP: this VM is $(hostname), not op-worker2; run step b first"; fi
```

Your SSH session freezes or closes. That's expected: the VM is moving to 10.0.1.22 and rebooting.

i. Check it on its own address (wait about a minute after the move):

```powershell
ssh -p 2222 root@127.0.0.1 "hostname; ip -4 -br addr show enp0s3; ip -br link show enp0s8; df -h /scratch /var/lib/containers; grep HugePages_Total /proc/meminfo; systemctl is-active crio"
```

Check:

- `op-worker2`, and `10.0.1.22/24`;
- `enp0s8` shows `UP` and `PROMISC`, with no address;
- `/scratch` is 80 G, and `/var/lib/containers` is 100 G;
- `HugePages_Total: 2048`;
- CRI-O is `active`.

When op-worker2 passes its check, all three nodes are on their own addresses. Go on to Phase 5.

PHASE 5

Kubernetes with kubeadm and Flannel

Result of this phase: a 3-node Kubernetes v1.36 cluster `rac-op`: the op-controlplane plus op-worker1 and op-worker2, all `Ready`, with Flannel as the pod network, and both workers labelled for the operator.

5.1 The kubeadm configuration (op-controlplane)

Blocks labelled `# op-controlplane` run in a root shell on the op-controlplane. From here on, keep one PowerShell window connected to it:

```powershell
ssh -p 2220 root@127.0.0.1
```

Check: the prompt says `[root@op-controlplane ~]#`. The same goes for `# op-worker1` (port 2221) and `# op-worker2` (port 2222) when a block needs them.

Why each part:

- `advertiseAddress` / `controlPlaneEndpoint` **10.0.1.20:** the API server listens on the public network, not on any other NIC.
- `criSocket` **CRI-O:** the runtime chosen in Phase 2.
- `podSubnet` **10.244.0.0/16:** Flannel's default. The doc's Grid response file names `eth0:10.244.0.0` as the public network inside the pods, so this range has to match.
- `dnsDomain` **cluster.local:** the SCAN name becomes `racnode-scan.rac.svc.cluster.local`.
- `allowedUnsafeSysctls`: the RAC pods set `kernel.shmall`, `kernel.shmmax`, `kernel.shmmni`, `kernel.sem` and some `net.*` settings for themselves. Kubernetes calls these "unsafe" and the kubelet rejects any pod that sets them, unless they're allowed here. These settings are namespaced: they affect only that pod, not

the node. The OCNE doc sets this as a kubelet flag; with kubeadm it belongs in the `KubeletConfiguration`, which every node picks up when it joins.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
cat >/root/kubeadm-config.yaml <<'EOF'
apiVersion: kubeadm.k8s.io/v1beta4
kind: InitConfiguration
localAPIEndpoint:
  advertiseAddress: 10.0.1.20
  bindPort: 6443
nodeRegistration:
  name: op-controlplane
  criSocket: unix:///var/run/crio/crio.sock
---
apiVersion: kubeadm.k8s.io/v1beta4
kind: ClusterConfiguration
clusterName: rac-op
controlPlaneEndpoint: 10.0.1.20:6443
networking:
  podSubnet: 10.244.0.0/16
  serviceSubnet: 10.96.0.0/12
  dnsDomain: cluster.local
---
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration
cgroupDriver: systemd
allowedUnsafeSysctls:
  - "kernel.shm*"
  - "kernel.sem"
  - "net.*"
EOF
```

5.2 Initialise the control plane (op-controlplane)

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubeadm init --config /root/kubeadm-config.yaml
mkdir -p ~/.kube && cp -f /etc/kubernetes/admin.conf ~/.kube/config
kubectl get nodes
```

Check: `Your Kubernetes control-plane has initialized successfully!`. The op-controlplane shows `NotReady` until there's a pod network in 5.3, which is expected.

5.3 Flannel, the pod network (op-controlplane)

What: Flannel gives every pod an `eth0` address in 10.244.0.0/16, and routes pod traffic between nodes over the 10.0.1.x network.

Why: Kubernetes needs a CNI network before any pod can start. In the RAC pods, `eth0` is the **public** network: the node names, VIPs and SCAN all resolve to `eth0` addresses.

This URL always fetches the newest Flannel release. The lab used the release current on 23 Oct 2026. If a newer one misbehaves, pick a specific version from Flannel's releases page instead of `latest`.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
kubectl -n kube-flannel rollout status ds/kube-flannel-ds
kubectl get nodes
```

Check: the op-controlplane turns `Ready` within a minute.

5.4 Join the workers

What: a join command with a fresh token, run on each worker. The PC fetches it from the op-controlplane and runs it on both workers, so there's nothing to copy by hand. The token in it expires after 24 hours.

PowerShell

```
$join = ssh -p 2220 root@127.0.0.1 "kubeadm token create --print-join-command"; $join
ssh -p 2221 root@127.0.0.1 $join
ssh -p 2222 root@127.0.0.1 $join
```

Each worker ends with `This node has joined the cluster`.

Back on the op-controlplane, label the workers and wait for them. The second label, `raccluster=raccluster01`, is the operator's: its `workerNodeSelector` places RAC pods only on nodes with this label.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl label node op-worker1 op-worker2 node-role.kubernetes.io/worker= --overwrite
kubectl label node op-worker1 op-worker2 raccluster=raccluster01 --overwrite
kubectl wait --for=condition=Ready node --all --timeout=600s
kubectl get nodes -o wide
kubectl get pods -A
```

Check:

- three nodes, all `Ready` and on `v1.36.x`, with INTERNAL-IP 10.0.1.20, .21 and .22;
- every system pod `Running`: coredns, etcd, kube-apiserver, kube-proxy, kube-flannel;
- `kubectl get nodes -l raccluster=raccluster01` lists op-worker1 and op-worker2.

If a worker shows the wrong INTERNAL-IP, the `--node-ip` setting from Phase 4 is missing.

5.5 kubectl from your PC (optional)

Why: so you can run `kubectl` from Git Bash on Windows without SSH-ing into op-controlplane first. The manual lab may already use `~/.kube/config` and host port 6443, so the operator lab gets **its own file**, `~/.kube/config-op`, and **its own port, 6444**. The two never overwrite each other.

PowerShell

```
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" natnetwork modify --netname k8s-public --port-forward-4 "k8sapi-op:tcp:[]:6444:[10.0.1.20]:6443"
```

Git Bash

```
mkdir -p ~/.kube
scp -P 2220 root@127.0.0.1:/etc/kubernetes/admin.conf ~/.kube/config-op
kubectl --kubeconfig "$(cygpath -m ~/.kube/config-op)" config set-cluster rac-op --server=https://127.0.0.1:6444 --tls-server-name=kubernetes
sed -i '/alias kop=/d' ~/.bashrc
echo "alias kop='kubectl --kubeconfig \"\$(cygpath -m ~/.kube/config-op)\"'" >> ~/.bashrc
grep -q MSYS_NO_PATHCONV ~/.bashrc || echo 'export MSYS_NO_PATHCONV=1' >> ~/.bashrc
source ~/.bashrc
kop get nodes
```

Why each line:

- `k8sapi-op` **on 6444:** the manual lab's forward uses host port 6443. A second rule can't use the same host port, so the operator lab's API server is reached on 6444; inside the lab it still listens on 6443.
- `--kubeconfig ~/.kube/config-op` **:** keeps the operator lab's credentials apart from the manual lab's `~/.kube/config`.
- `set-cluster rac-op` **:** kubeadm names the cluster entry after `clusterName` (`rac-op`), so that's the entry to point at 127.0.0.1:6444.
- `--tls-server-name=kubernetes` **:** the API server's certificate is for `kubernetes` and 10.0.1.20, not 127.0.0.1; this keeps certificate checking on.
- `kop` **:** short for `kubectl --kubeconfig C:/Users/<you>/.kube/config-op`. Use `kop` for the operator lab and plain `kubectl` for the manual lab. `cygpath -m` turns the Git Bash path into a Windows one, because `kubectl.exe` is a Windows program and `MSYS_NO_PATHCONV=1` stops Git Bash from converting `/c/Users/...` for it. Without that, kubectl can't find the file and falls back to `localhost:8080`.

Check: `kop get nodes` lists `op-controlplane`, `op-worker1` and `op-worker2`. `admin.conf` gives full control of the cluster; keep it out of anything you share.

The same in PowerShell: a `kop` function in your PowerShell profile. PowerShell passes Windows paths, so no conversion is needed.

PowerShell

```
if (-not (Test-Path $PROFILE)) { New-Item -ItemType File -Force -Path $PROFILE | Out-Null }
if (-not (Select-String -Path $PROFILE -Pattern 'function kop' -Quiet)) {
  Add-Content -Path $PROFILE -Value 'function kop { kubectl --kubeconfig "$env:USERPROFILE\.kube\config-op" @args }'
}
. $PROFILE
kop get nodes
```

Check: the three `op-*` nodes. If `. $PROFILE` says running scripts is disabled, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once and try again. For a whole session on one lab, `$env:KUBECONFIG = "$env:USERPROFILE\.kube\config-op"` points plain `kubectl` at the operator lab until the window closes.

PHASE 6

Multus and the RAC private networks

Result of this phase: pods can have two extra interfaces, `eth1` on 10.0.11.0/24 and `eth2` on 10.0.12.0/24, with addresses you choose. A test between the workers proves the interconnect works before any Oracle software is involved.

6.1 Why RAC needs Multus

A Kubernetes pod normally has exactly one network interface, `eth0`, from the cluster network (Flannel here). RAC needs more than that:

- **A public network:** client connections, the VIPs and SCAN. That's `eth0`.
- **A private interconnect:** dedicated, low-latency and not routed through the cluster network. Cache Fusion moves database blocks between the instances over it, and the cluster heartbeat runs on it. If the heartbeat is lost, Grid evicts a node.

Multus is a "meta" CNI plugin that attaches additional networks to a pod. Each extra network is described by a **NetworkAttachmentDefinition** (NAD). This lab uses `macvlan`, which gives the pod its own MAC address directly on the worker's `enp0s8` / `enp0s9`. Two networks give Grid's **HAIP** (highly available IP) two paths, so the interconnect survives losing one.

6.2 Install Multus (op-controlplane)

The thick-plugin variant runs a small daemon on every node. The URL follows Multus's main branch, as it did in the lab on 23 Oct 2026. For a repeatable build you can replace `master` with a release tag.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl apply -f https://raw.githubusercontent.com/k8snetworkplumbingwg/multus-cni/master/deployments/multus-daemonset-thick.yml
kubectl -n kube-system rollout status ds/kube-multus-ds --timeout=300s
```

Check: `daemon set "kube-multus-ds" successfully rolled out`.

6.3 The two RAC networks (op-controlplane)

Why each setting:

- **Namespace** `rac` **:** the operator looks the NADs up in the same namespace as the RAC database it creates, and its samples use `rac`. The manual lab used a separate `multus-rac` namespace; this is the one difference from it.
- **Names** `macvlan-conf1` **/** `macvlan-conf2` **:** the names the operator's `privateIPDetails` will refer to.
- `master: enp0s8` **/** `enp0s9` **:** the worker NICs prepared in Phase 4. The doc uses `ens5` / `ens6`, its own NIC names.
- `mode: bridge` **:** pods on the same parent NIC can reach each other.
- `host-local` **IPAM with a range of 140–150:** the doc's subnets and ranges.
- `"capabilities": {"ips": true}` **:** lets each pod request a **specific** address (10.0.11.141 for racnode1-0, .142 for racnode2-0). Without it, `host-local` ignores the request and picks addresses per node on its own, so both workers can hand out the same IP. The doc's NADs don't have this line; this is a delta.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl create namespace rac
cat <<'EOF' | kubectl apply -f -
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
  name: macvlan-conf1
  namespace: rac
spec:
  config: '{ "cniVersion": "0.3.1", "type": "macvlan", "master": "enp0s8", "mode": "bridge",
    "capabilities": { "ips": true },
    "ipam": { "type": "host-local", "subnet": "10.0.11.0/24", "rangeStart": "10.0.11.140", "rangeEnd":
"10.0.11.150" } }'
---
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
  name: macvlan-conf2
  namespace: rac
spec:
  config: '{ "cniVersion": "0.3.1", "type": "macvlan", "master": "enp0s9", "mode": "bridge",
    "capabilities": { "ips": true },
    "ipam": { "type": "host-local", "subnet": "10.0.12.0/24", "rangeStart": "10.0.12.140", "rangeEnd":
"10.0.12.150" } }'
EOF
kubectl -n rac get network-attachment-definitions
```

6.4 Prove the interconnect (op-controlplane)

What: one throwaway pod on each worker, each with `eth1` and `eth2` at chosen addresses. Then pings across.

Why now: if the private network doesn't work, Grid's installer fails much later with a far less obvious error. In the first build, this test is what exposed the promiscuous-mode problem. `NET_RAW` is added because CRI-O drops it by default and `ping` needs it.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
for n in 1 2; do o=$((147+n)); cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: nettest-w$n
  namespace: rac
  annotations:
    k8s.v1.cni.cncf.io/networks: '[{"name":"macvlan-conf1","interface":"eth1","ips":["10.0.11.$o/24"]},
{"name":"macvlan-conf2","interface":"eth2","ips":["10.0.12.$o/24"]}]'
spec:
  nodeSelector: { kubernetes.io/hostname: op-worker$n }
  containers:
  - name: net
    image: docker.io/library/busybox:1.36
    command: ["sleep", "infinity"]
    securityContext: { capabilities: { add: ["NET_RAW"] } }
EOF
done
kubectl -n rac wait --for=condition=Ready pod/nettest-w1 pod/nettest-w2 --timeout=300s
kubectl -n rac exec nettest-w1 -- ip -4 addr
kubectl -n rac exec nettest-w1 -- ping -c3 10.0.11.149
kubectl -n rac exec nettest-w1 -- ping -c3 10.0.12.149
kubectl -n rac exec nettest-w2 -- ping -c3 10.0.11.148
kubectl -n rac exec nettest-w2 -- ping -c3 10.0.12.148
```

Check:

- `nettest-w1` has `eth1` 10.0.11.148/24 and `eth2` 10.0.12.148/24;
- all four pings get 0% packet loss.

If you get 100% loss, check that `enp0s8` / `enp0s9` show `PROMISC` on both workers (Phase 4, step c on each worker), and that 3.2's check showed `Promisc Policy: allow-all` for both workers.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl -n rac delete pod nettest-w1 nettest-w2
```

PHASE 7

Install Oracle's database operator

Result of this phase: cert-manager and the Oracle Database Operator running in the cluster, watching namespace `rac`, with the extra permissions the RAC controller needs: reading nodes and the Multus networks, and creating the ASM PersistentVolumes itself.

What the operator is: a controller that runs in its own namespace, `oracle-database-operator-system`. It adds new object types to Kubernetes, `RacDatabase` among them. When you later apply one `RacDatabase` YAML, the operator creates everything the manual lab built by hand in its Phases 7–12: the PVs, the headless Services, the StatefulSets, the Grid and database installs and the database.

7.1 Get the operator's manifests (op-controlplane)

Why a copy of the repository: the install files are generated for the namespaces the operator should watch, and the RAC permission files live in the repository. The lab used the version current on 26 Oct 2026 (release v2.2.0). The block prints the exact commit, so you can note it.

`op-controlplane`

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
command -v git >/dev/null || dnf install -y -q git
rm -rf /root/oraop && git clone -q --depth 1 https://github.com/oracle/oracle-database-operator.git
/root/oraop
cd /root/oraop && git log -1 --format='commit %h, %cd'
ls oracle-database-operator-rbac.yaml oracle-database-operator-system.yaml scripts/generate-namespace-
install.sh rbac docs/rac/rbac
```

Check: a commit line, and the files and two `rbac` folders listed without errors.

7.2 cert-manager (op-controlplane)

Why: the operator checks every `RacDatabase` you apply through a webhook, and Kubernetes only talks to webhooks over TLS. cert-manager issues and renews that certificate. The version is the one the operator's README names.

`op-controlplane`

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl apply -f https://github.com/cert-manager/cert-manager/releases/download/v1.20.2/cert-manager.yaml
kubectl -n cert-manager rollout status deploy/cert-manager deploy/cert-manager-webhook deploy/cert-
manager-cainjector --timeout=300s
kubectl -n cert-manager get pods
```

Check: three `successfully rolled out` lines, and three cert-manager pods `Running` with `1/1`.

7.3 The operator, watching namespace rac (op-controlplane)

Why namespace-scoped: the operator's recommended mode. It only acts in the namespaces it's told to watch, here its own and `rac`, where the Multus networks from 6.3 already are. The script writes the matching RBAC and sets `WATCH_NAMESPACE`, so the two can't disagree.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
cd /root/oraop
scripts/generate-namespace-install.sh oracle-database-operator-system,rac
kubectl apply -f dist/install/oracle-database-operator-rbac.yaml
kubectl apply -f dist/install/oracle-database-operator-system.yaml
kubectl -n oracle-database-operator-system rollout status deploy/oracle-database-operator-controller-manager --timeout=600s
kubectl -n oracle-database-operator-system get pods -o wide
kubectl -n oracle-database-operator-system get deploy oracle-database-operator-controller-manager -o jsonpath='{range .spec.template.spec.containers[0].env[?(@.name=="WATCH_NAMESPACE")]}{.name}={.value}{"\n"}{end}'
```

Check:

- `successfully rolled out`, and the controller-manager pods `Running`;
- `WATCH_NAMESPACE=oracle-database-operator-system,rac`.

The first start pulls the operator image from `container-registry.oracle.com`, which can take a few minutes. If a pod stays in `ErrImagePull`, run `kubectl -n oracle-database-operator-system describe pod -l control-plane=controller-manager | tail -20` and paste the events.

7.4 The RAC controller's extra permissions (op-controlplane)

Why: the namespace-scoped install can only touch objects inside the watched namespaces. The RAC controller also needs a few cluster-wide ones:

FILE	LETS THE OPERATOR	NEEDED BECAUSE
`rbac/node-rbac.yaml`	read nodes	it picks workers by the `raccluster` label and builds NodePort connect strings
`rbac/persistent-volume-rbac.yaml`	read PersistentVolumes	it checks the ASM PVs
`docs/rac/rbac/pv-rbac.yaml`	create and delete PersistentVolumes	it creates the ASM PVs from the disk paths you list
`docs/rac/rbac/multus-rbac.yaml`	read NetworkAttachmentDefinitions	it attaches `macvlan-conf1` / `macvlan-conf2` to the RAC pods

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
cd /root/oraop
kubectl apply -f rbac/node-rbac.yaml -f rbac/persistent-volume-rbac.yaml -f docs/rac/rbac/pv-rbac.yaml -f docs/rac/rbac/multus-rbac.yaml
SA=system:serviceaccount:oracle-database-operator-system:oracle-database-operator-controller-manager
for c in "list nodes" "create persistentvolumes" "delete persistentvolumes" "list network-attachment-definitions.k8s.cni.cncf.io -n rac" "create statefulsets.apps -n rac" "create racdatabases.database.oracle.com -n rac"; do
  printf '%-62s %s\n' "$c" "$(kubectl auth can-i $c --as=$SA)"
done
kubectl get crd racdatabases.database.oracle.com
```

Check: every line ends in `yes`, and the `racdatabases.database.oracle.com` CRD is listed. A `no` names the permission that's missing; paste the output.

PHASE 8

The RAC slim image and the software

Result of this phase: Oracle's RAC slim image, `localhost/oracle/database-rac:19.3.0-slim`, on both workers, and the two 19.32 gold-image zips in `/scratch/software/stage` on both workers.

Why on both workers: the operator may start either RAC pod on either labelled worker, and each worker's CRI-O can only start images it has locally. The image never leaves the worker it was built on, so nothing needs a registry.

8.1 Build the slim image (op-worker1, then op-worker2)

What the slim image is: Oracle Linux 9 with the OS packages, users and groups Oracle needs, plus Oracle's own setup scripts, but **no** Oracle software. The operator starts this image and drives the scripts in it; they install Grid and the database from the zips in 8.2 into `/u01`, on the worker's disk. That's why the manual lab's image, built from the OCNE doc's short Containerfile, can't be used here: it doesn't have those scripts.

Why each line:

- `podman-docker`: Oracle's build script calls `docker`. This package makes `docker` run Podman, which writes into the same image store CRI-O reads (`/var/lib/containers`). `/etc/containers/nodocker` silences its "emulating docker" notice.
- **A sparse clone:** only Oracle's RAC folder of the large `docker-images` repository is downloaded.
- `-v 19.3.0`: the 19c folder, which the README says to use for all 19c builds, RUs such as 19.32 included. Its `Containerfile` uses Oracle Linux 9.
- `-i`: skip the checksum of install zips; the slim image doesn't contain any.
- `SLIMMING=true`: build without the Oracle software.
- `TMPDIR` **on** `/scratch`: Podman's temporary files go to the 80 GB disk instead of the small root disk.

The build downloads Oracle Linux packages and takes about 10–20 minutes. You can run it in op-worker1 and op-worker2 at the same time, in two windows.

`op-worker1, then op-worker2` (10-20 minutes each)

CONNECT FIRST `ssh -p 2221 root@127.0.0.1` ; for op-worker2: `ssh -p 2222 root@127.0.0.1`

```
dnf install -y -q git podman-docker && touch /etc/containers/nodocker
mkdir -p /scratch/image/tmp && rm -rf /scratch/image/docker-images
git clone -q --depth 1 --filter=blob:none --sparse https://github.com/oracle/docker-images.git
/scratch/image/docker-images
cd /scratch/image/docker-images && git sparse-checkout set
OracleDatabase/RAC/OracleRealApplicationClusters && git log -1 --format='docker-images commit %h, %cd'
cd OracleDatabase/RAC/OracleRealApplicationClusters/containerfiles
export TMPDIR=/scratch/image/tmp
./buildContainerImage.sh -v 19.3.0 -t localhost/oracle/database-rac:19.3.0-slim -i -o '--build-arg
SLIMMING=true' > /scratch/image/build.log 2>&1; echo "build exit $?"
tail -8 /scratch/image/build.log
podman images localhost/oracle/database-rac
crictl images | grep database-rac
```

Check, on each worker:

- `build exit 0`, and the log ends with `Oracle Database container Image for Real Application Clusters (RAC) version 19.3.0 is ready to be extended` and `--> localhost/oracle/database-rac:19.3.0-slim`;
- `podman images` and `crictl images` both list `localhost/oracle/database-rac` with tag `19.3.0-slim`. `crictl` seeing it proves CRI-O, and so Kubernetes, can start it.

If the build fails, the full log is `/scratch/image/build.log`; paste its last 40 lines (`tail -40 /scratch/image/build.log`).

8.2 Stage the gold-image zips (from the PC)

What: both 19.32 gold-image zips copied from `C:\Users\alpoo\Downloads\oracle-stage` to `/scratch/software/stage` on **both** workers, then checked against the PC's own checksums.

Why each part:

- **Both workers:** the operator installs from the stage directory of the worker each RAC pod runs on, and either pod may land on either worker. It mounts the directory as `hostSwStageLocation`.
- **Zipped, not unpacked:** the operator unzips them itself; the `RacDatabase` names them in `gridSwZipFile` and `dbSwZipFile`. Oracle's prerequisites accept a 19.28 or later gold image, and these are 19.32.
- **Mode 755:** the permissions Oracle's prerequisites ask for on the zips.
- `restorecon`: gives the files the `container_file_t` label set up in 4f, so the pods may read them under SELinux.
- **Checksums:** 9.6 GB through a NAT port forward is easy to cut short. `sha256sum -c` on each worker compares against the checksums taken on the PC; a truncated or damaged copy fails here instead of an hour into the Grid install.

The copy takes a few minutes per worker. The checksums take about a minute on each side.

Git Bash

```
cd ~/Downloads/oracle-stage
sha256sum oracle-gridhome-19.32.zip oracle-dbhome-19.32.zip | tee /tmp/zips.sha256
for p in 2221 2222; do echo "--- copying to port $p"; scp -P $p oracle-gridhome-19.32.zip oracle-dbhome-19.32.zip root@127.0.0.1:/scratch/software/stage/; done
for p in 2221 2222; do echo "--- checking port $p"
  ssh -p $p root@127.0.0.1 "hostname; cd /scratch/software/stage && chmod 755 oracle-*.zip && restorecon -F oracle-*.zip && ls -lhZ oracle-*.zip && sha256sum -c" < /tmp/zips.sha256
done
```

Check: for `op-worker1` and `op-worker2`, both zips listed with `-rwxr-xr-x`, about 4.8 G and `container_file_t`, and two lines ending in `OK`. A `FAILED` line means that copy is damaged: run the `scp` line again for that port, then the check.

Next: Phases 9–12

After Phase 6 the operator lab is at the same point as the manual lab before its Phase 7, with three deliberate differences the operator asks for: the node directories `racnode1` / `racnode2` on both workers (4f), the worker label `raccluster=raccluster01` (5.4), and the Multus networks in namespace `rac` (6.3). The rest of the worker set-up (kernel settings, SELinux module, `allowedUnsafeSysctls`, hugepages, clock source) was already in the golden image and the kubeadm configuration.

The phases below follow Oracle's Oracle Database Operator for Kubernetes (release v2.2.0, RAC controller `RacDatabase`, API `database.oracle.com/v4`). Phases 7 and 8 are above; the rest will be written out the same way.

PHASE	WHAT	FROM THE OPERATOR DOCS
9. Secrets	`ssh-key-secret` (keys `ssh-privkey`, `ssh-pubkey`); `db-user-pass` (password encrypted with OpenSSL, typed at a prompt, never written to the guide)	`create_kubernetes_secret_for_*.md`
10. ASM disks	no PVs by hand: the `RacDatabase` lists the shared disks by their `/dev/disk/by-id/...` names and the operator creates the PVs	`asmDiskGroupDetails` in `racdb_prov.yaml`
11. The `RacDatabase`	one YAML: 2 nodes, `racnode` prefix, `/scratch/rac/cluster01`, the two NADs, the worker label, CPU/memory/hugepages sized for 22 GB workers, `DATA` disk group, the gold-image zips, then `kubectl apply` and follow `/tmp/orod/oracle_db_setup.log` until "ORACLE RAC DATABASE IS READY TO USE"	`racdb_prov.yaml`, `racdb_prov_memory_cpu.yaml`, `racdb_prov_hugepages.yaml`
12. Access from Windows	the operator's NodePort option, port forwards, and the same SCAN test as the manual lab	`provisioning_oracle_rac_db_with_node_port.md`, `database_connection.md`

Open points:

- **Which image (settled).** The manual lab's image can't be reused: it was built from the OCNE doc's short Containerfile, and the operator drives the setup through the scripts in Oracle's own RAC image (it follows `/tmp/orod/oracle_db_setup.log`). Oracle doesn't publish the slim image on its registry, so Phase 8 builds it on each worker with `buildContainerImage.sh -v 19.3.0 -i -o '--build-arg SLIMMING=true'` (Oracle Linux 9 base).
- **Sizing.** Oracle's prerequisites name 60 GB of RAM per worker and samples with 30 GiB pods. The lab's workers have 22 GB, so the `RacDatabase` sets resources explicitly, as the manual lab did (12 GiB, 3 CPU, 4 GiB hugepages).
- **Platform.** Oracle lists OKE, OpenShift, OLCNE 1.9+ and Minikube as tested. A plain kubeadm cluster with CRI-O isn't on that list, so expect lab-specific workarounds, as in the manual lab.

Built from the lab's build log and scripts (rac-k8s-poc). Values are from the author's lab; change them to match yours.